

(Python)

chapter 1

1st python program:-

```
print("HelloWorld")
```

.py is the extension of python:-

Module

It is file containing code written by somebody else.

e.g. pip install pyjokes

Pip

pip is the package manager for python. You can use pip to install a module on your system.

Types of modules

- 1) Built-in modules
- 2) External modules

Python as

Work:-

calculator

open window

terminal:-

↳ we can use python as calculator
by writing "python" +

↳ This open REPL
(Read Evaluate Print loop)

Comments ((#) single line comment)

(single line
comment) ⇒ #

(for ignoring
lines:)

(multiple line
comment) ⇒ """

..."

(for give better
explanation:)

DAT: _____

Scanned with CamScanner

Primarily these are following data types in python.

- integers (1, 2, 3, 4)
- floating point number (2.11, 7.98)
- String ("awais")
- Booleans (True, False)
- None

#None

(To mark that variable contains nothing)

Rules:-

- A variable name can contain alphabet, digits and undercores.
- A variable name can only start with an alphabet and underscores.
- A variable name can't start with a digit.
- No while space is allowed to be used inside a variable name.

Examples:-

harry, one8, seven, _seven etc.

(Operators in python)

Followings are some common operators in python:-

- a) Arithmetic operator : $+$, $-$, $*$, $/$ etc.
- b) Assignment operator : $=$, $+=$, $-=$ etc.
- c) Comparison operator : $==$, $>$, $>=$, $<$, $!=$
- d) Logical operator : and , or , not

a) Arithmetic operator

$a = 34$
 $b = 4$
 $c = a + b$
 $print(c)$

$$\begin{array}{ccccc}
 & & & \text{operator} & \\
 7 & + & 4 & = & 11 \\
 \swarrow & & \searrow & & \swarrow \\
 \text{operands} & & & & \text{result}
 \end{array}$$

b) Assignment operator.

```

a = 4 - 2    # Assign 4-2 in a
print a

b = 6

b += 3    # increment value of b by 3
print(b)  and then assign it to b.
  
```


c) $\hookrightarrow$ comparison operator
(give output in boolean form)

$d = 5 < 4$ $d = 5 >= 5$ $d = 5 < 5$
`print(d)` `print(d)` `print(d)`

d) Logical operator
 $e = \text{True or false}$
`print(e)`

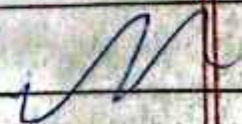
Logic operator

`print not(false)`

`True`

Multicursor
functionality of
VS code

By click on ALT
when select
they all are select
not edited at once



Type function and Type casting

(Type() function is used to find the datatype of a given variable in python)

```
a = 31
```

```
t = type(a)
```

```
print(t)
```

```
class <'int'>
```

```
a = 31.2
```

```
t = type(a)
```

```
print(t)
```

```
class <'float'>
```

* A number can be converted into string and vice versa (if possible):-

↳ By using functions

```
a = "31.2"
```

```
t = type(a)
```

```
print(t)
```

```
class <'str'>
```

if i want b as it a but type should be float then

```
b = float(a)
```

(Input function)

"allows user to take input from keyboard as a string"

A = input("enter name") #if a is "harry", the user enter harry

Note

it is important to note that output of input is always a string, (even if a number is entered)

"Harry" + "Coder"

HarryCoder

"1" + "2"

12

(chapter 3)

(String)

- ↳ String is a datatype in python
- ↳ String is a sequence of character enclosed in quotes
- ↳ Three ways to write:

a = 'harry'

b = "harry"

c = """harry"""

(String is immutable
("one created, its
value doesn't
change"))

String

Slicing

(String can be sliced to getting a part of string).

Name ⇒ **HARRY** ⇒ Length: 5

0	1	2	3	4	} indexing (numbering)
(-5)	(-4)	(-3)	(-2)	(-1)	

(For length of String)

```
name = "Avaais"
```

```
length = len(name)
```

```
print(length)
```

(5)

*) Slicing Syntax

$$\text{Sname} = \text{[ind_start : ind_end]}$$

variable first index is included last index is not included

*) Slicing with skip value

```
word = "amazing"
```

$$\text{word}[1 : 6 : 2]$$
 no. of skipping values

mzn

(String functions)

(case sensitive)

(a) `len("Harry")`

return length of string

(b) `string.endswith("rry")`

("tells whether string ends with rry or not")

(c) `string.count("c")`

("counts total occurrence of any character")

(d) `string.capitalize()`

("capitalize first character of given string") (only at first word not after space)

e) upper/lower

`string.upper()` → convert all to upper case

`string.lower()` → convert all to lower case

f) title

`string.title()` make first letter of each word capitalize

g) Swapcase

`String.swapcase()`

(changes upper to lower and vice versa)

h) Strip()

`String.strip()` (remove spaces from both ends)

i) lstrip()

`String.lstrip()`

((remove spaces from left))

j) rstrip()

`String.rstrip()`

((remove spaces from right))

k) replace()

`text.replace("Java", "Python")`

replace one string with another

l) find()

(Return first index of

`text.find("substring")` substring)

↳ (return -1) if not find)

m) index()

Like find(), but return error if not find.

`string.index("t")`

n) split()

"convert string to list"

`text.split()`

o) join()

join list items to string

`"separator".join('string')`

p) is alpha function

↳ (Check if all characters are letter).

`String.isalpha()`

q) isdigit function

Check if all characters are digits:

`String.isdigit()`

r) **isalnum()**

(check if letters are letters and numbers)

`String.isalnum()`

s) **isspace()**

Check if string contains only spaces

`String.isspace()`

t) **center()**

width of new string
character

`String.center(width, fillchar)`

`String.center(15, "*")` `++++A++S++++`

u) **zfill()**

(Add 0) to left until

String reaches specified width)

`text = "25"`

`text.zfill(5)`

`00025`

v) **casefold()**

~~text~~ Like `lower()`, but powerful :-

`text.casefold()`

DATE: _____

at beginning (DAY)

(w) **removeprefix()**
text.removeprefix(prefix)

at ending

(x) **removesuffix()**
text.removesuffix(suffix).

(Escape Sequence character)

- Sequence of character after backslash "\".
- (Escape characters comprises of more than one character but represent one character when used within the string)

example \n, \t, \', \", etc

single quote
|
new line tab back slash.

- i) New line: (\n)
- ii) Horizontal tab: (\t)
- iii) Backslash (\)
- iiii) Single quote (\')
- iiiii) Double quote (\")
- iiiiii) Backspace (\b)
- vii) Carriage return (\r)
- (viii) Alert (\a)
- (ix) form feed (\f)
- (x) Vertical tab (\v)

f string

if we want variable value in the string then we can use f string

```
name = "Ali"
```

```
print (f"Hello {name}")
```

(Lists are mutable)

DATE: _____

DAY: _____

(Chapter 4)

List AND TUPLES

((python lists are container used to store a set a set of values of any datatype)).

```
friends = ["apple", "akash", 7, false]
```

str() int() Bool()

(List Indexing)

"A list can be indexed just like a string"

(List Slicing)

same as string listing

(List methods)

append() → append (add at end of list) ^{only one}

sort() → sort list

insert(index, item) → insert item

pop() delete (n) → ^{at a specific position} index

extend() → add multiple items ^{position type:}

clear() → emptied list

index() → return index of 2nd occurrence of item

count() (how many times an item appears)

copy() (return a shallow copy of the list)

Tuple

→ Immutable, not edited once created

a = () # empty tuple

a = (1,) # tuple with one element

Tuple

Method

(only two build-in methods):-

count()

return number of times a specified value appeared in tuple.

index()

return index of first occurrence of the specified value.

tuple_name.index count (value)

tup_name.index (startend, value)

(Tuples

Method)

(Operations)

Concatination (+)

Join two tuples into one

$t_1 + t_2$

Repetition (*)

Repeat tuple n times

$t_2 * 2$ $\rightarrow$ n times

Membership (in)

check if value exist

value in tuple name

in a tuple

Membership (not in)

check if value doesn't
exist in a tuple

value not in

tuple-name

Indexing

Access an element
using its index

tuple-name[index]

Slicing

Return position
of tuple

tuple-name (start: end
: step)

DATE: _____

PAGE: _____

Max.

~~length~~

return largest
element

`max (tuple-name)`

Min

~~Maximum~~

return smallest
element

~~Minimum~~ length.

return ~~sum~~ total number of
elements `len (tuple-name)`.

Sum

`sum (tuple-name)`

Sum of numerics value:

Dictionary & Sets

• Dictionary is the collection of key value pairs.

```
a = { "key" : "value",  
      "harry" : "code",  
      "marks" : "100",  
      "list" : [1, 2, 9]  
}
```

```
a["key"] # print "value"
```

```
a["list"] # print [1, 2, 9]
```

Properties

↳ it is unordered

↳ is mutable

↳ it is indexed

↳ ^{NOT} contain duplicate keys

(Methods)

clear()

remove all the items from the dictionary.

[dict.clear()]

copy()

returns a shallow copy of the dictionary.

[dict.copy()]

fromkeys()

(["", "], 0 ----)

Create a dictionary with specified keys and one value.

[dict.fromkeys(keys, value)]

get()

(" ")

return value of key without causing an error if key doesn't exist.

[dict.get(key, default)]

items()

Return all key-value pair as tuple object.

[dict.items()]

keys()

return all keys in the dictionary.

[dict.keys()]

pop()

remove the specified key and return its value. (" ")

[dict.pop()]

popitem()

removes and return the last inserted key-value pair.

[dict.popitem()]

setdefault()

returns value of key; if absent, insert a key with a default value.

[dict.setdefault(key, value)]

update()

Adds or update key-value pairs from another dictionary

[dict.update(other dict)]

values()

return all values in dict.

[dict.values()]

Sets

Set is a collection of non-repetitive elements.

`S = set()` # no repetition allowed

`S.add(1)`

`S.add(2)` # or `set={1,2}`

Sets Methods

(i) `add` (add one element to set)

(ii) `update` (Add multiple elements from list, tuple or set.)

(iii) `remove` (remove a specified elements from set. If doesn't exist, it raise a `KeyError`)

(iv) `discard` (remove a specified elements. unlike `remove()`: it doesn't give any error if element is not present).

(v) `Union` (return a new set containing all unique elements from b. sets)

(vi) `Intersection` (return elements that are common to both sets)

(vii) `pop` (remove and return first element)

(viii) `clear` (remove all elements.)

(ix) difference Method

(return elements that are present in first set but not in second set)

Chapter 6

Conditional

Expression

- ↳ Sometimes we want to play PUBG on our phone if the Day is Sunday
- ↳ Sometimes we order ice-cream online if the day is Sunny.
- ↳ Sometimes we go hiking if our parents allow.
- ↳ All these are decisions which depend on a condition being met
- ↳ In python programming, we must be able to execute instructions on condition(s) we met.
- ↳ That is what condition are for!

(IF ELSE AND ELIF IN PYTHON)

↳ if else and elif statements are a multiway decision taken by our program due to certain condition in our code:

Syntax

```
if (condition): # condition is True
    print("yes")
elif (condition1): # if condition1 is True
    print("no")
else: # otherwise
    print("may be")
```

(Relational operation)

↳ used to evaluate conditions inside the if-statement. Some examples are:

`== ;`

`> ;`

`< = ;`

(Logical operators)

AND : true if both operands are true

OR : true if one of both operands is true.

NOT : negates

(ELIF-Clause) (Multiple if)

`a = int(input("Enter your age : "))`

1 `if(a % 2 == 0):`
`print("a is even")`

2 `if (a >= 18):`

`print("You are above age of consent")`

`elif (a < 0):`

`print("You are entering invalid age")`

`else: print("You're below age of consent")`

We have to see in this case, which if-elif are connected.

1st if is separate 2nd if is ladder structure

Note

a) There can be any number of elif statements.

b) last else execute only if upper all conditions are false.

Loops

4 Sometimes, we want to repeat a set of statements in our program. For instance: Print 1 to 1000.

Types

•) while loop

•) for loop

While loop

while(condition): ((block keeps continue until the condition is true))
#Body of loop.

↳ (In while loop, condition is checked first - if it evaluates to true body of the loop is executed) otherwise not!)

↳ If loop is entered, process of [condition check and execution] is continued until the condition becomes false).

Example

```
i = 0
while(i < 5):
    print("Harry")
    i += 1
```


For loop

↳ Used to iterate through a sequence like string, list or tuple;

Range function:

`range()` is used in python to generate a sequence of numbers.

(we can also specify the start, stop and step-size as follow:)

`range(start, stop, step-size)`

("step-size is usually not used with `range()`")

* For loop with else

(An optional else can be used with a for loop if code is to be executed when loop exhausts.)

Example :-

```
l = [1, 7, 8]
```

```
for item in l:
```

```
    print(item)
```

```
else: print('done') # it is printed when loop exhausted
```

Break statement:-

is used to come out of the loop when encountered. It instructs the program to exit the loop now.

```
for i in range(100):
```

```
    print if (i == 34):
```

```
        break # Exit loop right
```

```
        print(i)    now.
```

Continue statement

Used to stop current iteration of loop and continue with next one. It instructs the program to skip the iteration.

```
for i in range(100)
```

```
    if (i == 34):
```

```
        continue
```

```
    print(i).
```


Pass statement

(pass is null statement
in python)

It instructs "do nothing".

```
for i in range(648):
```

if i want to stop here and want
to execute another one,

Like,

```
for i in range(648):
```

write there

```
i = 0
```

```
while (i < 45):
```

```
    print(i)
```

```
    i += 1
```

it gives error

→ pass

for this

Chapter 8

functions

- ↳ A function is a group of statement performing a specific task.
- ↳ When a program gets bigger in size and its complexity grows, it gets difficult for a program to keep track on which piece of code is doing what!
- ↳ A function can be reused by the programmer in a given program any number of:

Syntax

```
def func1():  
    print("hello")
```

This function can be called any number of times, anywhere in program.

↳ function call

Whenever we want to call a function, we put name of the function followed by the parenthesis as follow

`func1()` # This is a function call.

↳ function definition

Part containing exact set of instructions which are executed during the function call.

Types

Builtin
function

User defined
function

• Already present in Python.

• Example:
`len()`, `print()`,
`range()`

• defined by the user.

• `function()`

↳ Function with Arguments

↳ A function can accept some value it can work with. We can put these values in the parenthesis.

↳ A function can also return value as shown below:

```
def greet(name)
    gr = "hello" + name
    return gr
a = greet("harry")
# a now contain
"hello harry"
```

parameters

```
def greet(name):
    print("Hello", name)
    greet("Amar")
```

function call

argument

default argument.

we can have a value as a default as default argument in the function.

```
def greet (name = "stranger"):
```

```
    greet ()
```

```
    greet ("harry")
```

Recursion

is a function which calls itself. It is directly use a mathematical formula as a function:-

Formulae Example

$\text{factorial}(n) = n \times \text{factorial}(n-1)$.

In program

```
def factorial(n):
```

```
    if (n == 0 or n == 1):
```

```
        return 1
```

```
    else:
```

```
        return n * factorial(n-1)
```

Factorial (5)

5 × factorial (4)

5 × 4 × factorial (3)

5 × 4 × factorial (3)

5 × 4 × 3 × factorial (2)

5 × 4 × 3 × 2 × factorial (1)

5 × 4 × 3 × 2 × 1 = 120

DATE: _____

DAY: _____

Chapter No 9

File I/O

volatile:-
data is temporary
and is lost when
program terminates,
or power is turned off

Non-volatile:-
not lost when
program terminates

- Q Random Access memory is volatile,
all its contents are lost once a program
terminates.

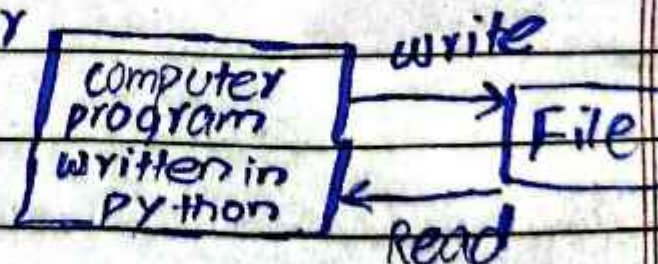
In order to persist data forever, we
use files

- Q A file is data stored in storage device.
A python program can talk to file by
reading content from it and writing
content to it.

Persist:

In order to persist
data forever, files are used. After a program terminates,
data remains save for
a long time, even

Programmer



RAM = volatile

HDD = Non volatile

Types of files

- ↳ Text files (.txt, .c)
- ↳ Binary files (.jpg, .dat)

*) python has a lot of functions for reading, updating and deleting files.

opening a file

Python has an opening function() for open files. it takes 2 parameter filename and mode.

```
# open("filename", "mode of opening (read mode by default)")  
open("this.txt", "r")
```

*) Reading a file

```
f = open("this.txt", "r")
```

```
text = f.read()
```

```
print(text)
```

```
f.close()
```


(other methods to read file)

we can also use `f.readline()` function
to read one full line at a time.

`f.readline()` it returns list
of lines

Modes of opening a file

- r) open for reading
- w) open for writing
- a) open for appending
- rb) will open for read in binary mode.
- rt) will open for read in text form.

Suppose

`file.txt`

a) Read

Hello World
Welcome to Python

```
f=open("file.txt", "r")
```

```
data=f.read
```

```
print(data)
```

```
f.close()
```

Hello word

Welcome to Python

b) write

used to write data to file.

```
st = "Hi! rana; You're amazing"  
f = open("file.txt", "w")  
f.write(st)  
f.close()
```

if file.txt already had something, its old content will be deleted.

Now it contain

Hi! rana; You're amazing

c) Append (add new content at the

```
f = open("file.txt", "a")
```

if file.txt contain:

Hello Python

```
f.write("\nwelcome to Python")
```

```
f.close()
```

old content is not
deleted

Hello Python
welcome to Python

d) `rt` (read and write; use when you want to read and write in the same file)

```
f = open("file.txt", "rt")  
print(f.read())  
f.write("\nNewline")  
f.close()
```

e) `rb` (read binary)

Used to read files in binary form, such as images, PDF, audio

```
f = open("image.jpg", "rb")  
data = f.read()  
print(data)  
f.close()
```

Output like:

`b'\xff\xd8\xff\xe0`

DATE: _____

DAY: _____

read_text(r)

used to read a normal text file.

```
f = open("file.txt", "r")
```

```
data = f.read()
```

```
print data
```

```
f.close()
```

r is basically normal text reading mode, while r means already text mode by default.

with statements

The best way to open and close the file automatically is with, the with statement

↳ not need to close file;

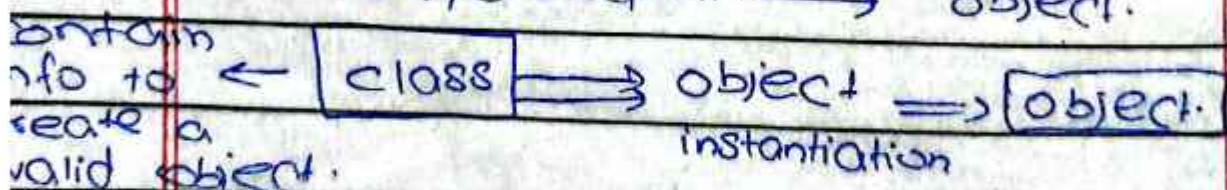
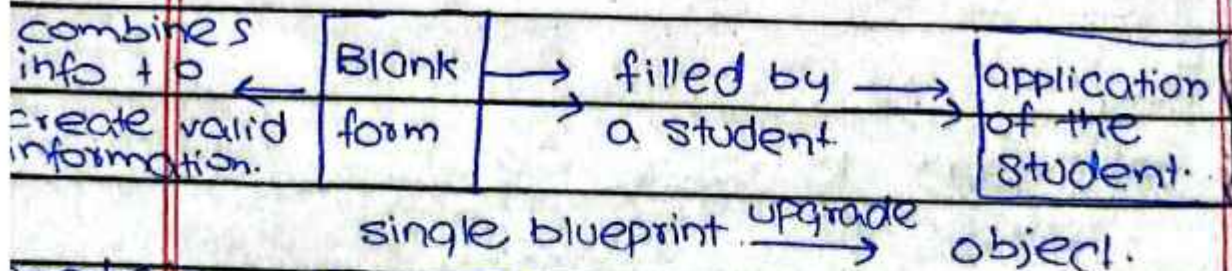
CHAPTER 10

((Object oriented Programming))

Solving a problem by creating object is one of the most popular approaches in programming. This is called OOP.

Class

A blueprint for creating an object.



Syntax

class Employee : # class name is written in pascal
methods & variables.

OBJECT

An object is an instantiation of the class.

When class is defined, a template(info) is defined.

Memory is allocated only after object instantiation.

Modelling a problem in oops:-

we identify followings in our problems.

- * Noun → class → Employee
- * Adjective → Attribute → name, age, salary
- * Verb → Methods → getSalary(), increment()

CLASS ATTRIBUTES

An attribute that belongs to the class rather than a particular object.

e.g.:- class Employee:

company = "Google" # Specify to each

harry = Employee() # ^{class} object instantiation

harry.company

Employee.company = "Youtube" # ^{changing} _{class} attribute

Instance

Attribute

An attribute that belong to the instance(object). Assume class from previous example:-

```
harry = Employee()
```

```
harry.name = "harry"
```

```
harry.salary = '30k' # Add instance attributes.
```

Instance vs class

Instance attributes take preference over class attributes during assignment and retrieval.

It check following:

1) Is per attribute present in object?

2) Is attribute present in class?

→ Self parameter

Self refers to the instance of class.
It is automatically passed when a function call from an object.

```
harry.getSalary() # here self is harry  
# equivalent to Employee.getSalary(harry)
```

↳ function getSalary() is defined as:

```
class Employee:  
    company = "Google"  
    def getSalary(self):  
        print("Salary is not there")
```

→ Static Method

Sometime we need a function that doesn't use the self parameter. we can define a static method like this.

```
@staticmethod  
def greet()  
    print("Hello User")
```

#decorator to mark
greet as static

-init- () Constructor

↳ -init- () is the special method which is first run as soon as the object is created.

↳ -init- () method is also known as constructor.

↳ It takes self - argument and can also take further arguments.

for example:-

```
class Employee:
```

```
    def -init- (self, name):
```

```
        self.name = name
```

```
    def getsalary (self):
```

```
        ....
```

```
        ....
```

```
harry = Employee("Harry")
```

Chapter 7

Inheritance

It is a way of creating a new class from an existing class.

Syntax:-

```
class Employee: #Base class
    #code

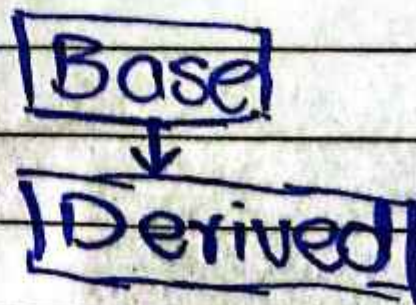
class Programmer(Employee) #derived
                                class.
    #code
```

- ↳ we can use methods/attributes of 'Employee' in "Programmer" object.
- ↳ Also, we can write ^{over-} or add new attributes and methods in "Programmer" class

Types

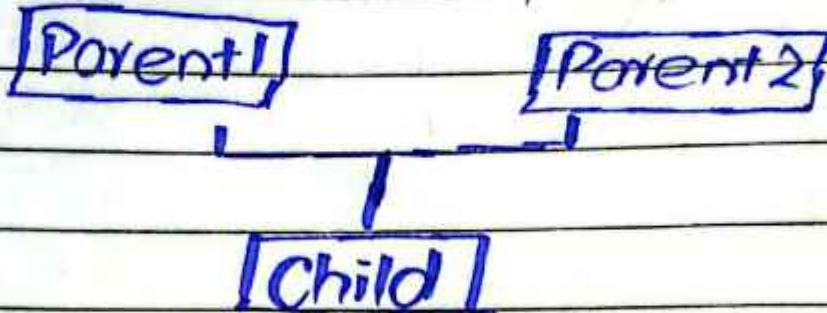
Single Inheritance

occurs when child class inherits only a single parent class.



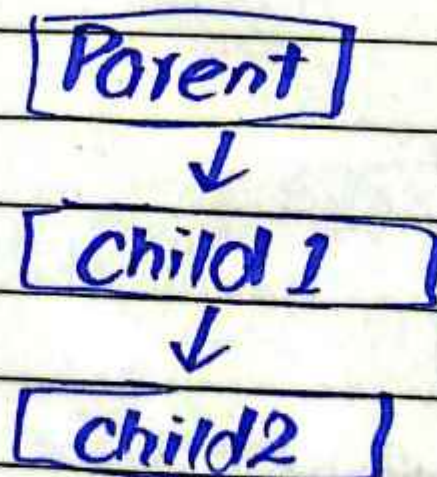
Multiple Inheritance

↳ occurs when the child class inherits from more than one parent classes.



Multilevel Inheritance

occurs when a child class becomes a parent for another child class.



Super() Method

It is used to access the methods of a super class in the derived class

```

> Super().__init__()
# __init__() calls constructor of Base class

```

Class Method

It is a method which is bound to class and not the object of class.

@classmethod decorator is used to create a class method.

```

C → @classmethod
    def(cls, p1, p2):

```

(allows to use method like an attribute)

Property decorators

```

class Employee:

```

```

    @property

```

```

    def name(self)

```

```

        return self.name

```

if e = employee() is

an object of employee

class; we can print

e.name to print

ename by internally

calling name() function

operator overloading

@Getter AND @setter

↳ Method name with '@property' decorator is called getter method.

↳ We can define a function + @name.setter decorator like this;

```
@ name.setter
```

```
def name(self, value)
```

```
    self._name = value
```

object overloading

* Operator in Python can be overloaded using dunder methods

* These methods are called when a given operator is used on the objects

* Operators in Python can be overloaded by using followings:

DATE: _____

DAY: _____

$P_1 + P_2 \# p1. - \text{add} - (p2)$

$P_1 - P_2 \# p1. - \text{sub} - (p2)$

$P_1 * P_2 \# p1. - \text{mul} - (p2)$

$P_1 / P_2 \# p1. - \text{truediv} - (p2)$

$P_1 // P_2 \# p1. - \text{floordiv} - (p2)$

$\text{str}() \#$ used to get what gets
displayed upon calling $\text{str}(\text{obj})$

$\text{len}() \#$ used to set what gets
displayed upon called $\text{len}() \text{ or } \text{len}(\text{obj})$

chapter 12 (Advanced Python)

★ (newly added features in python)

a) Walrus operator

walrus operator (`:`) introduced in python 3.8. allows you to assign values as a part of an expression. This operator, named for its resemblance to the eyes and tusks of walrus, is officially called the "Assignment or expression".

```
if n:=len([1,2,3,4,5]) > 3:  
    print(f"list is too long {n} elements,  
expected <= 3")
```

In this example, `n` is assigned with value of `len([1, 2, 3, 4, 5])` and then use in comparison within if statement

Type definition:-

Type hints are added using the colon (:) syntax for the variables and $\rightarrow$ syntance for function return types

```
# variable type hint
```

```
age: int = 25
```

```
# function type hint
```

```
def greeting(name: str)  $\rightarrow$  str:
```

```
    return f"Hello, {name}!"
```

```
# usage
```

```
print(greeting("Alice"))
```

Advanced type hint

* Python `typing` module provides more advanced type hints, such as list, tuple, dict and union.

* You can import List, Tuple, Dict types from typing like this:

```
from typing import List, Tuple, Dict, Union
```

Syntax:-

Source code

$\rightarrow$ make code self-documenting

$\rightarrow$ allow developers to understand DS at glance.

Match case

↳ Python 3.10 introduced the match statement, which is similar to the switch statement found in other programming languages.

↳ Basic syntax of match

Statement involves making a variable against several cases using the case keyword.

{Code: Source code}

Dict. merge and update operators

New operators `|` and `|=` allow for merging and updating dictionaries.

```
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}
merged = dict1 | dict2
print(merged)
```


You can now use multiple context managers in a single `with` statement more cleanly using the parenthesised context managers

```
with (  
    open('file1.txt') as f1,  
    open('file2.txt') as f2  
):  
    # Process file
```

Exception (doesn't crash program). Handling

There are many build-in functions exceptions which are raised in python when something goes wrong.

Exception handling can be handled using try statement. The code that handles the exception is written in except clause.

```
try:  
    # code which might through exception  
except handling as e:  
    print (e)
```


When the exception is handled, the code flow continues without program interruption. We can also specify the exception to catch like below:

```
try:  
    # code  
except ZeroDivisionError:  
    # code  
except TypeError:  
    # code  
except:  
    # code
```

All other
exception are
handled here.

Raising Exception

We can raise custom exception using the raise keyword in python.

```
(To show error) age = -5  
if age < 0:  
    raise ValueError("Age can't  
be -ve")
```

Try with else clause

Sometimes we want to run a piece of code when try was successful.

```
try:
```

```
# some code
```

```
except:
```

```
# some code
```

```
else:
```

```
# code
```

! # This is executed only if try was successful !

(when try runs successfully, then $\Rightarrow$ it will go to else block)

Try with finally

python offers a

"finally" clause which ensure execution of a piece of code irrespective of execution.

```
try:
```

```
# some code
```

```
except:
```

```
# some code
```

```
finally: # some code:-
```


(if `__NAME__` == `'--MAIN--'`
in python)

`'__name__'` evaluates to the name of the module in python from where the program is ran. if the module is being run directly from cmd line the `'__name__'` is set to the string `"__main__"`. Thus, this behavior is used to check whether the module is run directly or imported to another file.

```
if __name__ == "__main__":  
    print("Running directly")
```

(`__name__`) used to identify while the code belong to that class or imported from somewhere.

Global keyword

◦ 'Global' keyword is used to modify the variable outside of the current scope.

```
x = 10  
def change():  
    global x  
    x = 20
```

Enumerate function

'Enumerate' function adds counter to an iterable and returns it.

```
list1 = ["Harry", "Rohan", "Shobham"]  
for i, item in enumerate(list1):  
    print(i, item)
```

List comprehensions

It is an elegant way to create list based on existing lists.

```
List 1 = [1, 7, 12, 11, 22]
```

```
List 2 = [item for item in list1 if  
           item > 8]
```


Chapter 13 (Advanced Python 02)

Virtual Environment

An environment which is same as the system interpreter, but is isolated from other python environment on the system.

Installation

To use virtual environment, we write `[pip install virtualenv]`

We can also create a new environment using:

`[virtualenv myprojectenv]`

↳ next step after creating the virtual environment is to activate it.

↳ We can now use this virtual environment as a separate python installation.

Pip freeze command

'pip freeze' returns all packages installed in the given Python environment along with the version.

```
{ pip freeze > requirements.txt }
```

above command creates a file named 'requirements.txt' in same directory containing the output "pip freeze".

we can distribute the same environment/file to other users, and they can recreate the same environment using:

```
{ pip install -r requirements.txt }
```

Lambda function

function created using an expression using "lambda" keyword.

Syntax:

{ lambda arguments:
expression }

```
Square = lambda x: x*x
```

```
Square(6)
```

```
sum = lambda a,b,c: a+b+c
```

```
sum(1,2,3)
```


join Method (String)

Create String from iterable objects.

```
l = ["apple", "mango", "banana"]  
result = ", and, ".join(l)  
print(result)
```

*) The above line will return:
" apple , and, mango , and, banana "

Format method (String)

formats the value inside the string into a desired output.

```
template.format(p1, p2, ...)
```

Syntax

```
"{} is a good {}".format  
("harry", "boy")
```

```
" {1} + is a good {0} ".  
format("harry", "boy")
```


Map

map applies a function to all items in an input_list.

Syntax: `map(function, input_list)`

Filter

filter creates a list of item for which the item function returns true.

Syntax: `list(filter(function))`

Reduce

reduce applies a rolling computation to sequential pair of elements.

```
from functools import reduce  
val = reduce(function, list)
```

if the function computes sum of two numbers and the list is `[1, 2, 3, 4]`.